

1. Wstęp

Do porównania wybrałem listę dotyczącą metod zespołowych, w której rozwiązywałem problem klasyfikacji poziomu otyłości na zbiorze Estimation of Obesity Levels (UCI). Zadanie jest dla mnie interesujące, ponieważ łączy klasyczne metody zespołowe (drzewa, bagging, boosting, Random Forest, głosowanie i stacking) z istotnym wątkiem jakości danych, czyli rozpoznaniem wycieku celu. Zbiór ma siedem klas o naturalnym porządku, od niedowagi do otyłości trzeciego stopnia, co czyni go dobrym materiałem do porównania różnych podejść.

2. Wybór pracy konkursowej

Jako konkurs odpowiadający temu problemowi wybrałem Playground Series S4E2, którego zadaniem jest wieloklasowa klasyfikacja poziomu otyłości. Jest to ten sam typ problemu co w mojej liście, z tą różnicą, że dane konkursowe są syntetyczne, wygenerowane przez model uczenia głębokiego na podstawie tego samego oryginalnego zbioru UCI, którego ja używałem, i są znacznie liczniejsze.

Głównym punktem odniesienia jest praca easy-peasy: EDA, XGBoost is all you need, ponieważ jest czytelna i mimo to realizuje wszystkie pięć etapów na wysokim poziomie. Porównywałem również inne, mniej informacyjne rozwiązania.

3 Eksploracyjna analiza danych

Nasze analizy są zbliżone. Po obu stronach pojawiają się statystyki opisowe, rozkłady cech, korelacje oraz analiza względem klas. Moja eksploracja jest w kilku miejscach co

najmniej równie głęboka, a niekiedy głębsza. Wartości odstające analizuję boxplotami w podziale na klasy, co jednocześnie pokazuje rozrzut cechy w każdej klasie, podczas gdy praca konkursowa robi boxploty globalne. Kategorie analizuję względem targetu, licząc rozkład klas wewnątrz kategorii, a nie tylko rozkłady brzegowe. Dodatkowo rozpoznaję wyciek BMI, czego prace konkursowe nie wychwytyją, ponieważ działają na danych syntetycznych z rozmytymi progami (czyli u nich to nie są wycieki).

Jedyne, co notebook Visual EDA ma, a czego u siebie nie mam, to skonsolidowana tabela przeglądowa zbioru, łącząca w jednym miejscu typ cechy, liczbę braków, duplikaty i liczbę wartości unikalnych. U mnie te informacje są rozdzielone na osobne wywołania. Jest to według mnie o wiele czytelniejsze rozwiązanie.

4 Preprocessing

Podstawowy preprocessing mamy wspólny. Oboje stosujemy kodowanie one-hot cech kategoryalnych oraz łączymy rzadką kategorię CALC Always z Frequently, ponieważ pojedyncza obserwacja w treningu jest bezużyteczna i potrafi rozjechać kolumny względem zbioru testowego. Różnica polega na tym, że praca easy-peasy idzie dalej. Tworzy cechy pochodne o sensie dziedzinowym, na przykład poziom aktywności fizycznej z różnicy FAF i TUE oraz nawyki żywieniowe z iloczynu FCVC i NCP, a cechy CALC i CAEC koduje dodatkowo porządkowo, zachowując ich naturalną kolejność, której one-hot nie przekazuje. Inżynieria cech pochodnych jest głównym, czego u siebie nie zrobiłem, i jest realnym kierunkiem wzmocnienia. Robiona jest też walidacja adversaryjna, która sprawdza czy zbiór treningowy i testowy pochodzą z tego samego rozkładu (

u siebie pracuje na jednym zbiorze dzielonym testowo więc to zbędne)

5 Optymalizacja hiperparametrów

Strojenie zaczynamy podobnie, od ustawiania parametrów ręcznie i sprawdzania wyników. Praca konkursowa nazywa ten etap wprost strojeniem ręcznym i traktuje go jako punkt wyjścia, dokładnie tak jak ja w swojej liście przeszukuję ręcznie kombinacje współczynnika uczenia i liczby estymatorów. Różnica polega na tym, że po tym etapie praca konkursowa przechodzi do automatycznej optymalizacji biblioteką Optuna z samplerem TPE, strojąc kilkanaście parametrów naraz, gdzie każda próba oceniana jest w walidacji krzyżowej, a następnie trenuje finalny model z najlepszymi parametrami na całości danych. Cały proces jest uruchomiony i widoczny, w przeciwieństwie do wielu prac z tego konkursu, które proces strojenia komentują i wklejają gotowy słownik parametrów.

Krokiem, który mógłbym u siebie dodać, jest właśnie ta druga faza, czyli automatyczna optymalizacja po ręcznym rozpoznaniu zakresów, oraz finalny trening na całości danych po wyborze parametrów. Strojenie ręczne nie jest błędem, lecz naturalnym pierwszym krokiem, który zawęża przestrzeń, którą potem Optuna przeszukuje dokładnie.

6 Uczenie modelu

Oboje porównujemy modele i analizujemy błędy. W swojej liście buduję komitet czterech modeli przez głosowanie i stacking, łącząc je z równymi wagami, i obserwuję, że komitet nie pobił najlepszego pojedynczego modelu, ponieważ słabsze modele zaniżały predykcje silniejszych. Praca konkursowa trenuje XGBoost z walidacją krzyżową, przeprowadza analizę błędów wykazującą, że mylą się klasy sąsiadujące, czyli graniczne

stopnie otyłości, co stwierdzam także u siebie, a następnie łączy XGBoost z LightGBM, aby wygładzić granice tych klas. Blending dwóch silnych modeli gradientowych jest podejściem, którego w swoim komitecie z równymi wagami nie zastosowałem, a które adresuje dokładnie ten problem klas granicznych.

Praca testuje również dwa nietypowe podejścia. Pierwsze to ujęcie problemu jako regresji, uzasadnione porządkowym charakterem targetu, ponieważ regresja rozumie odległość między klasami i traktuje pomyłkę o jedną klasę jako mniejszą niż o kilka. Co istotne, podejście to wyszło empirycznie słabsze od klasyfikacji, accuracy spadło z około 0,90 do 0,88, co jest dobrą ilustracją tego, że nawet sensownie uzasadniona zmiana podejścia wymaga weryfikacji na danych, a trudność z klasami granicznymi leży w samych cechach, nie w sposobie ujęcia problemu. Drugie to eksperymentalna strategia multi output tree z XGBoost 2.0, w której jedno drzewo przewiduje wszystkie klasy naraz, wykorzystując ich powiązanie. Tę technikę odnotowuję, lecz jako eksperymentalną i poza zakresem listy świadomie jej nie przenoszę.

Najważniejszą cechą gradient boostingu, której ta praca uczy w kontekście danych tabelarycznych, jest to, że stanowi on mocny domyślny wybór. Stosowane przeze mnie metody zespołowe ze scikit-learn realizują tę samą ideę, lecz w słabszej i wolniejszej implementacji niż XGBoost czy LightGBM. Wybór konkretnej implementacji zależy od danych, ale samą tę rodzinę warto traktować jako punkt wyjścia.

7 Ewaluacja modelu

Autor ewaluuje modele na walidacji krzyżowej, generując predykcje out-of-fold dla całego zbioru i licząc na nich `classification_report` z metrykami per klasa, czyli precyzją, czułością i F1, oraz macierz pomyłek. Ocenę tę przeprowadza po każdym trenowanym modelu, dzięki czemu może porównywać podejścia na podstawie zmierzonych wyników. Na tej podstawie stwierdza na przykład, że podejście regresyjne daje słabszy wynik niż klasyfikacja, oraz że model najczęściej myli klasy sąsiadujące, co prowadzi go do decyzji o łączeniu modeli.

Dodatkowo praca konkursowa dokłada warstwę wyjaśnialności przez wartości SHAP. SHAP nie jest miarą skuteczności, lecz narzędziem interpretacji, które pokazuje wkład każdej cechy w predykcję, osobno dla każdej klasy, ujawniając kierunek i siłę wpływu cechy na decyzję modelu. Stosuje się je po to, aby sprawdzić, czy model opiera decyzje na sensownych cechach, a nie na wycieku, co buduje zaufanie potrzebne do wdrożenia i pomaga zdiagnozować, co poprawić. W moim przypadku SHAP wprost wskazałby dominację cech BMI i Weight, co spina się z moim głównym wnioskiem o wyciek. Tej warstwy interpretacji w mojej ewaluacji nie ma, więc jej włączenie jest realnym uzupełnieniem.

8 Wnioski zbiorcze

Etapy różni przede wszystkim cel. Praca konkursowa optymalizuje wynik na ukrytym zbiorze testowym i kończy przygotowaniem zgłoszenia, podczas gdy moja lista zmierza do oceny i zrozumienia modelu na własnym zbiorze z etykietami.

O wyniku decydują głównie cechy Height, Weight i BMI, co potwierdza analiza SHAP. Duży wpływ mają też inżynieria cech oraz dobór i strojeniu modelu. Zmiana podejścia z klasyfikacji

na regresję nie pomogła, co wskazuje, że trudność z klasami granicznymi tkwi w samych danych.

Do swojej listy przeniósłbym przede wszystkim dwie rzeczy: automatyczną optymalizację Optuną zamiast ręcznej, oraz inżynierię cech pochodnych. Uzupełniając rozważyłbym blend dwóch silnych modeli i analizę SHAP. Dodatkowo wiem, że modele gradient boosting dla problemów tabularycznych to standard.

<https://www.kaggle.com/code/lucamassaron/easy-peasy-eda-xgboost-is-all-you-need#4.0-Modelling>

<https://www.kaggle.com/code/ddosad/ps4e2-visual-eda-lgbm-obesity-risk/notebook>